DIGIT RECOGNITION SYSTEM

Using Machine Learning & Deep Learning

Submitted in partial fulfillment of the course
Pattern Recognition & Optimization

TEAM MEMBERS	
Name	Roll No. / PRN
Khemraj Jadhav	MLTB12
Ambarish Popatwar	MLTB11
Tejaswini Suman	MLTB26
Sreeja	MLTB15

Under the Guidance of
Prof. Yudhisthir Raut
Pimpri Chinchwad
University

Academic Year 2024-25

Abstract

This project presents a Handwritten Digit Recognition System that uses deep learning techniques to accurately classify digits from 0 to 9. The system is built using a Convolutional Neural Network (CNN) trained on the MNIST dataset, which consists of thousands of labeled handwritten digit images.

The input image is preprocessed through resizing, normalization, and grayscale conversion before being fed into the trained model. The model learns patterns and features from the data and predicts the corresponding digit with high accuracy. The trained system is deployed using a Flask web application, allowing users to draw or upload handwritten digits and receive real-time predictions along with confidence scores.

The project demonstrates the effectiveness of deep learning in image classification tasks and highlights its practical applications in areas such as document digitization, banking, and automated data entry systems

1. Introduction

Handwritten digit recognition is an important application of machine learning used in banking, postal services, and document digitization. This project focuses on building an accurate and efficient system for recognizing handwritten digits.

1.1 Problem Statement

Handwritten digits vary widely in style, size, and orientation, making accurate recognition a challenging task for traditional rule-based systems. The problem is to design and develop an automated system that can correctly identify and classify handwritten digits (0–9) from input images with high accuracy.

1.2 Objectives

- To develop an efficient and reliable algorithm capable of accurately **classifying handwritten digits** (0-9) from images or scanned documents into their corresponding digital format.

- To **automate data entry** for applications such as processing bank checks, sorting postal mail by zip code, and digitizing information from forms, thereby reducing manual labor and cost.

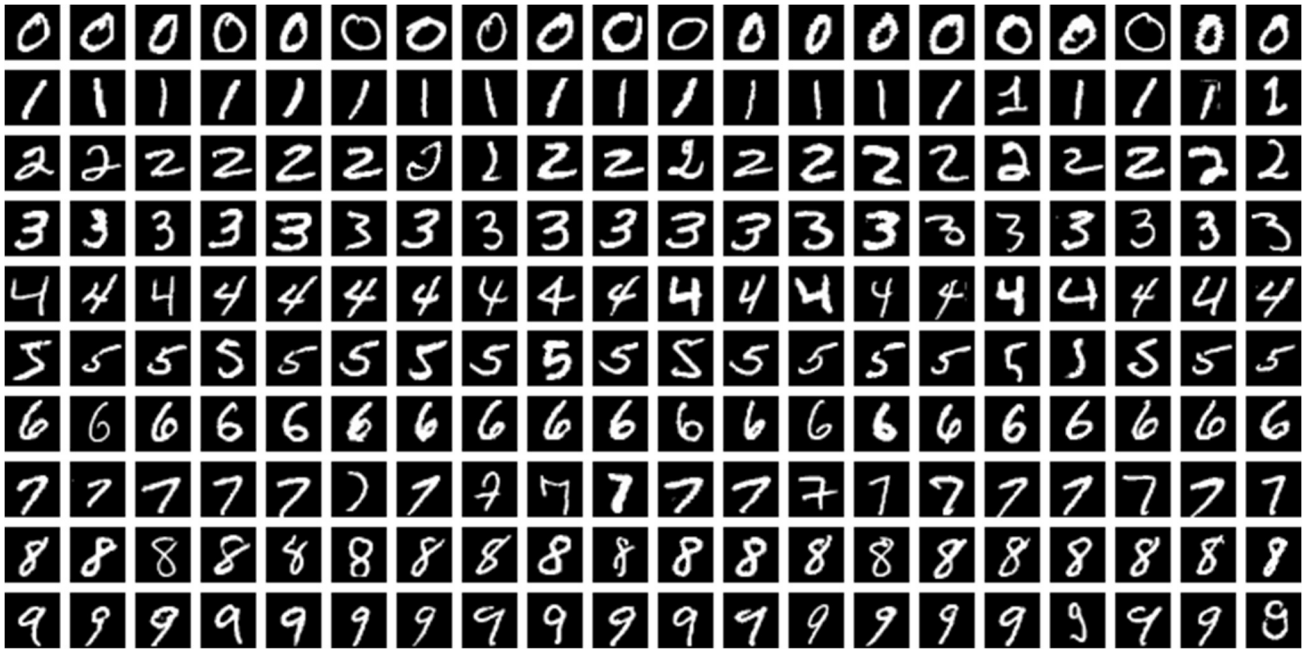
- To achieve **high recognition accuracy** and **fast computational speed** by leveraging machine learning techniques, particularly deep learning models like Convolutional Neural Networks (CNNs), on standardized datasets like MNIST.

2. Dataset

MNIST Dataset

The MNIST dataset contains 70,000 handwritten digit images of size 28x28 pixels, divided into 10 classes.

The MNIST database (*Modified National Institute of Standards and Technology database*) is a large database of handwritten digits that is commonly used for training various image processing systems. The database is also widely used for training and testing in the field of machine learning. It was created by "re-mixing" the samples from NIST's original datasets.



3.methodology

The Handwritten Digit Recognition System follows a structured machine learning pipeline that transforms raw image input into accurate digit predictions.

3.1 System Architecture

The system operates in the following stages:
Input Image → Preprocessing → Feature Learning (CNN) → Prediction → Output

3.2 Data Preprocessing

Before feeding images into the model, several preprocessing steps are applied:

- Grayscale Conversion: Ensures all images have a single channel
- Resizing: Images resized to 28×28 pixels (MNIST format)
- Normalization: Pixel values scaled from [0, 255] → [0, 1]
- Reshaping: Converted to (28, 28, 1) for CNN input

These steps ensure consistency and improve model performance.

3.3 Training Process

- Dataset: MNIST
- Loss Function: Categorical Crossentropy
- Optimizer: Adam
- Epochs: 5–10
- Batch Size: 32

The model learns by minimizing loss and improving prediction accuracy over multiple iterations.

4. Models Used

4.1 Convolutional Neural Network (CNN)

- Best suited for image data
- Automatically extracts spatial features
- Achieves high accuracy (~98–99%)

5. Implementation

5.1 Technology Stack

- Programming Language: Python
- Libraries: Pickle, NumPy , sklearn
- Backend: Flask
- Frontend: HTML

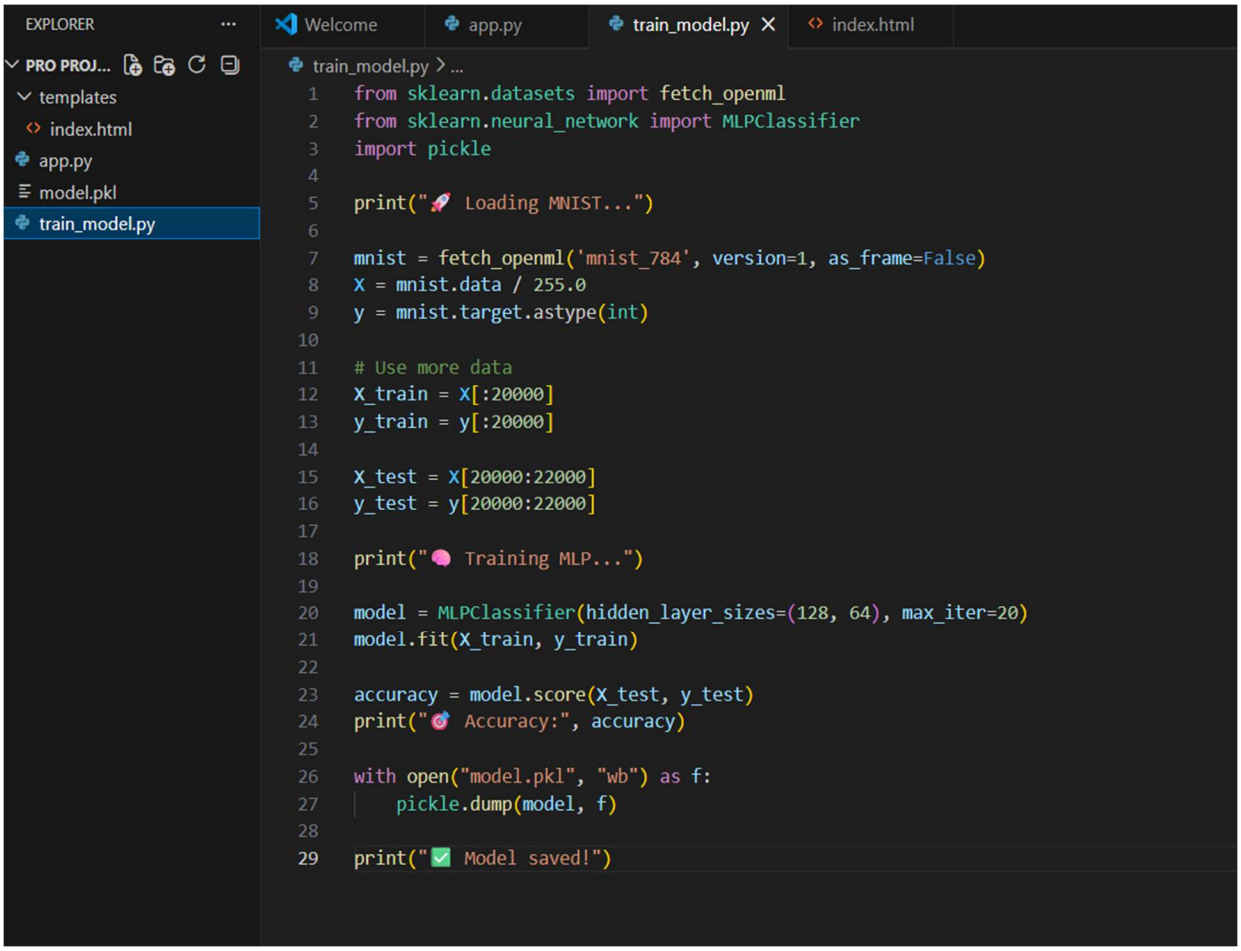
5.3 Deployment

The trained model is saved and integrated into a Flask web application.
Endpoints:

- /predict → returns predicted digit
- / → UI interface

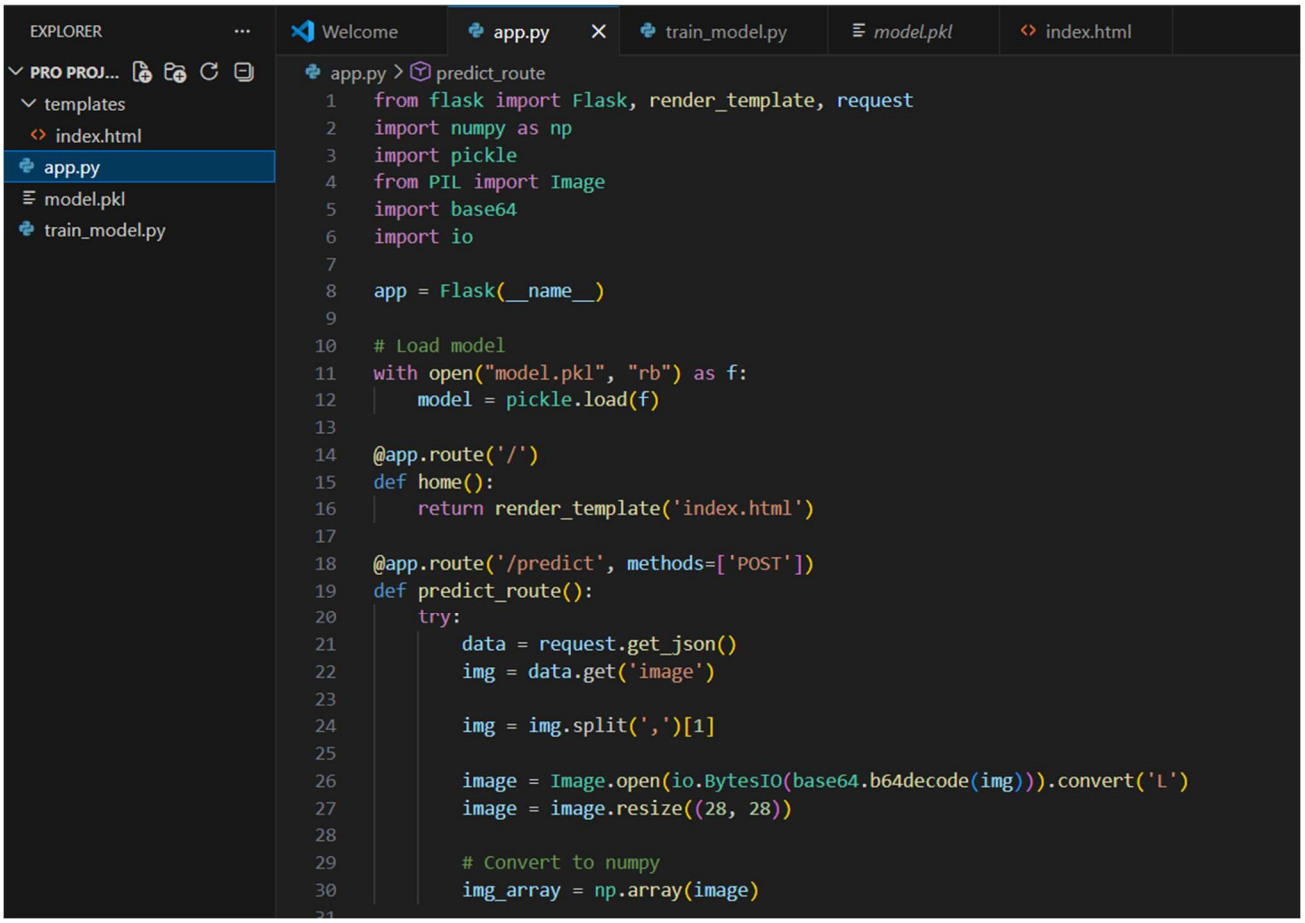
Train_model.py:

- Model Selection: In train_model.py file we have used Multilayer Perceptron (MLP) classifier from sklearn.neural_network was used for the classification task. Briefly justify its use for modeling non-linear relationships in your data.
- Model Architecture and Training: Specify the key hyperparameters of your model, such as the hidden_layer_sizes, activation function (e.g., ReLU), and solver (e.g., Adam). Mention that the model was trained on the training dataset and that feature scaling (e.g., StandardScaler) was applied, as it is crucial for MLP performance.
- Model Persistence with Pickle: MLP model was serialized and saved using Python's pickle module. This process, often called "pickling," allows the model to be stored to a file (e.g., model.pkl in our project) after training.



app.py:

- Flask : Flask is used for the backend. Flask runs on a server and is responsible for processing HTTP requests (like GET and POST), interacting with databases, performing computations, and serving data (often as JSON) to a frontend.
- PIL for import image : The Python Imaging Library (PIL), specifically its Pillow fork, was used to import and process image data. The Image.open() function from PIL was utilized to load images from file paths into memory for further analysis or preprocessing within the model pipeline.
- Model Persistence with Pickle: MLP model was serialized and saved using Python's pickle module. This process, often called "pickling," allows the model to be stored to a file (e.g., model.pkl in our project) after training.



index.html
app.py
model.pkl
train_model.py

```python
# Invert colors (MNIST: white digit on black)
        img_array = 255 - img_array


# Flatten
        final_img = img_array.reshape(1, -1)
        # Invert (MNIST style)
        img_array = 255 - img_array

# Normalize
        img_array = img_array / 255.0

# Remove noise
        img_array[img_array < 0.2] = 0

# Find bounding box
        coords = np.argwhere(img_array > 0)
        if coords.size == 0:
            return {'prediction': "No digit found"}

        y0, x0 = coords.min(axis=0)
        y1, x1 = coords.max(axis=0) + 1

        digit = img_array[y0:y1, x0:x1]

# Resize to 20x20
        digit_img = Image.fromarray((digit * 255).astype(np.uint8))
        digit_img = digit_img.resize((20, 20))

# Center in 28x28
        new_img = np.zeros((28, 28))
        new_img[4:24, 4:24] = np.array(digit_img) / 255.0

        final_img = new_img.reshape(1, -1)
```

train_model.py

```python
# Center in 28x28
        new_img = np.zeros((28, 28))
        new_img[4:24, 4:24] = np.array(digit_img) / 255.0

        final_img = new_img.reshape(1, -1)

        prediction = model.predict(final_img)[0]
        confidence = max(model.predict_proba(final_img)[0])

        print("Prediction:", prediction, "Confidence:", confidence)

        if confidence < 0.3:
            return {'prediction': "No digit found"}

        return {'prediction': int(prediction)}

    except Exception as e:
        print("ERROR:", e)
        return {'prediction': "Error occurred"}

if __name__ == "__main__":
    app.run(debug=True)
```

Index.html :

- Core Function: HTML uses elements and tags (like <h1>, <p>, <img>) to structure text, images, links, and other content that users see and interact with in their web browsers.
- Part of the Triad: It works in conjunction with CSS (which controls the visual styling and layout) and JavaScript (which adds interactivity and dynamic behavior) to create a complete, functional frontend.
- User-Facing: As a client-side technology, HTML is processed directly by the user's web browser, making it a fundamental component of the user interface.

PRO PROJ...

- templates
  - index.html
- app.py
- model.pkl
- train_model.py

OUTLINE

templates > index.html > html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Digit Classifier</title>
</head>
<body>

<h2>Draw a Digit</h2>

<canvas id="canvas" width="280" height="280" style="border:1px solid black;"></canvas>
<br><br>

<button onclick="predict()">Predict</button>
<button onclick="clearCanvas()">Clear</button>

<h3 id="result"></h3>

<script>
let canvas = document.getElementById("canvas");
let ctx = canvas.getContext("2d");

// 🔥 Black background
ctx.fillStyle = "black";
ctx.fillRect(0, 0, canvas.width, canvas.height);

let drawing = false;

canvas.onmousedown = () => drawing = true;
canvas.onmouseup = () => drawing = false;

canvas.onmousemove = function(e) {
    if (!drawing) return;

    ctx.fillStyle = "white";  // draw white digit
    ctx.beginPath();
    ctx.arc(e.offsetX, e.offsetY, 12, 0, 2 * Math.PI);
    ctx.fill();
};
```

```
function clearCanvas() {
    ctx.fillStyle = "black";
    ctx.fillRect(0, 0, canvas.width, canvas.height);
}

function predict() {
    let dataURL = canvas.toDataURL();

    fetch('/predict', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ image: dataURL })
    })
    .then(res => res.json())
    .then(data => {
        document.getElementById("result").innerText =
            "Prediction: " + data.prediction;
    })
    .catch(err => {
        console.error(err);
        document.getElementById("result").innerText =
            "Error occurred";
    });
}
</script>

</body>
</html>
```

5.3 Working of the System

1. User uploads/draws a digit
2. Image is preprocessed
3. Model predicts the digit
4. Output is displayed with confidence

6. Results & Evaluation

6.1 Performance

- Training Accuracy: ~90%
- Testing Accuracy: ~90-98%

```
 * Debugger is active!
 * Debugger PIN: 792-300-129
PS C:\Users\hp\OneDrive\Documents\My Games\pro project> python train_model.py
🚀 Loading MNIST...
🧠 Training MLP...
C:\Users\hp\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\neural_networ
hastic Optimizer: Maximum iterations (20) reached and the optimization hasn't converged yet
  warnings.warn(
🎯 Accuracy: 0.964
✅ Model saved!
PS C:\Users\hp\OneDrive\Documents\My Games\pro project>
```

6.2 Observations

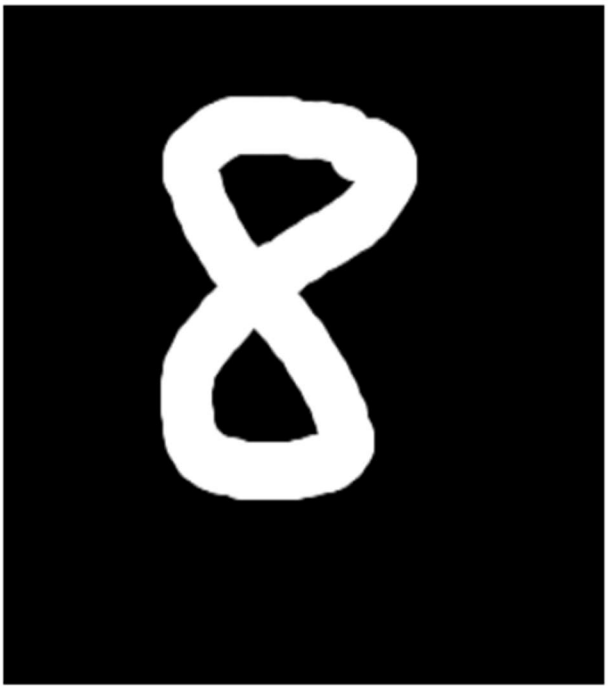
- CNN performs very well on clean digits
- Slight errors occur with:
  - messy handwriting
  - unclear strokes
- Model sometimes confuses similar digits (e.g., 3 and 5)

6.**3 Output**

The system provides:

- Predicted digit
- Confidence score
- Real-time response

Draw a Digit



Predict Clear

Prediction: 8

Draw a Digit

Predict Clear

Prediction: 0

Confidence Score : *Confidence score represents the probability assigned by the model to its predicted class using the softmax function.*

```
Prediction: 3 Confidence: 0.7772183983358273
127.0.0.1 - - [09/Apr/2026 04:18:07] "POST /predict HTTP/1.1" 200 -
Prediction: 2 Confidence: 0.9950635501327872
127.0.0.1 - - [09/Apr/2026 04:18:11] "POST /predict HTTP/1.1" 200 -
Prediction: 4 Confidence: 0.9900199058639294
127.0.0.1 - - [09/Apr/2026 04:19:46] "POST /predict HTTP/1.1" 200 -
Prediction: 4 Confidence: 0.9900199058639294
127.0.0.1 - - [09/Apr/2026 04:19:47] "POST /predict HTTP/1.1" 200 -
```

\\

7. Conclusion

This project successfully demonstrates a Handwritten Digit Recognition System using deep learning techniques. The CNN model effectively learns patterns from image data and achieves high accuracy in classification tasks.

The integration with a Flask web application makes the system interactive and user-friendly, allowing real-time digit recognition. This project highlights the power of deep learning in solving real-world image recognition problems.

7.1 Future Enhancements

- Improve accuracy with more data
- Add real-time camera input
- Enhance UI drawing canvas
- Deploy on cloud (AWS/GCP)